

Deep Learning Assignment 1

INFOMDLR — June 27, 2026 – Team 21

1st Bas de Blok
std. nr. 0934348

2nd Nicolae-Radu Olariu
std. nr. 0545341

3rd Charlie Choffat
std. nr. 9655301

4th Joris Heemskerk
std. nr. 5201284

Abstract—Time-series forecasting is a common problem. This paper attempts to apply RNNs, LSTMs, and transformers to predict a laser measurement signal. A custom heuristic was used to tune the hyperparameters. The best performing RNN achieved an MAE of 35.08 and an MSE of 2129, which was a significant improvement compared to the baseline with an infinite error.

Index Terms—time-series forecasting, rnn, lstm, transformer.

I. INTRODUCTION

The classification of brain states from neuroimaging data is a common problem in neuroscience and clinical research. The goal of this project is to classify the different cognitive state of a patient - resting, motor, working memory or math and story task - from magnetoencephalography (MEG) recordings using machine learning techniques. The measurements are sequential signals from 248 magnetometer sensors, thus, turning the task into a supervised classification problem.

In this paper, we applied RNNs [1], LSTMs [2], and transformers [3] to the dataset. A heuristic approach was used to approximate the optimal hyperparameters. The dataset was expanded using interpolation on individual signal components. These separate signal components were extracted and used as individual features, which were evaluated on the same models. Lastly, we report and discuss our results in terms of MAE and MSE, and compare them to a baseline model.

II. USED MODELS

Recurrent neural networks (RNNs) are very well suited for time-series forecasting tasks because of their ability to maintain an internal state while processing ordered inputs. Recurrent links were introduced to provide neural networks with dynamic memory, in order to allow previous inputs to influence later predictions [1]. Long Short-Term Memory (LSTM) networks expand this concept by using memory cells and gates to better manage longer temporal dependencies [2]. The more recently popular transformer models use an attention mechanism to learn and capture patterns across the context window [3]. Therefore, RNNs, LSTMs, and transformers, are suitable options for one-step-ahead prediction of the laser measurement signal.

III. METHODS

The training dataset consists of 1000 samples and the test set 200, each with a single input feature. The training values range from 2 to 255, with a mean of 59.9 and a standard deviation of 46.9. A visualisation of the dataset can be found in Figure 1.

The dataset was split into a train and validation partitions with a 80% / 20% split, such that the validation partition matches the size of the test set. To validate performance across all parts of the signal, the validation data was randomly sampled. Due to computational constraints, no cross-validation was used, though we recognise this would have given more accurate validation metrics. All data was normalised using the training partition. The data was then denormalised before calculating performance metrics.

A simple feed-forward neural network was developed to serve as a baseline for comparison with more advanced architectures. The baseline model uses three past time steps as input, has one hidden layer with three neurons, and a single output neuron to predict the next value in the sequence. This small and compact setup provides a low complexity reference point for evaluating our more advanced model architectures.

Regarding the hyperparameters, we tuned the window size (how many data points are present in each training sample), the hidden size (size of the hidden layers in the models), the number of layers in the models, the learning rate, the batch size, the stride (stepsize by which the data points are sampled), and the weight decay for the used adam optimiser. For each hyperparameter, a range of possibly optimal values was found by trying multiple values and manually looking for the minimum and maximum parameter values where the model might optimally perform. Between these ranges, 5 equally spaced values were selected. For this selection, both RNNs and LSTMs were analysed simultaneously, as these models benefit from similar hyperparameters. Transformers were not included, due to time and resource constraints. The values can be found in Table I. In Appendix A, we show our experimental results that led to the selection of these ranges. All models were trained for 100 epochs, as the loss indicated all models converged well before.

Due to resource constraints, we were unable to perform a grid search for all these values. Instead, we independently tuned each parameter, using the local optimal of the other hyperparameters. We reflect on this limitation in section V.

For evaluation, we used Mean Average Error (MAE) and Mean Squared Error (MSE). These metrics indicate how far the predictions deviate from the ground truth. Appendix B describes the details on these metrics. Throughout the tuning process, the MSE was used as the main deciding factor, as it penalizes larger errors more heavily than the MAE, making poor predictions more visible.

TABLE I: Hyperparameter configurations evaluated. Bold text, grey cells and underlined text highlight the best performing values for RNN, LSTM and transformer respectively.

	Win. size	Hid. size	Layers	Lr rate	Batch size	Stride	Weight decay
V ₁	30	40	<u>1</u>	1e-4	2	1*	1e-7
V ₂	40	60	<u>2</u>	5e-4	4	4	<u>1e-6</u>
V ₃	50	80	3*	1e-3	6	7	1e-5*
V ₄	60*	100*	4	1.5e-3*	8	10	1e-4
V ₅	70	120	5	2e-3	10*	13	1e-3

*Locally found optima.

A. Individual Signal Extraction

When analysing the data, we noticed that it appeared to consist of multiple distinct signals. We elaborate on how we found these in Appendix C. Specifically, we found seven distinct signals that appear to be interleaved with a set interval. We extracted these signals and turned them into individual features. We trained the most optimal models on this new dataset. However, extracting the signals reduces the number of data samples by a factor of seven. To compensate for this, we add additional data points for each individual signal using linear interpolation. We upsampled only our train and validation datasets by a factor of seven, restoring its length to match the original dataset.

IV. RESULTS

Table I shows the five candidate values for each of the seven hyperparameters, along with the locally optimal values identified through our tuning process for the RNN and LSTM. Tables IIa, IIb, and IIc report the MAE and MSE for the RNN, LSTM, and Transformer models for all hyperparameter configurations. The LSTM consistently outperformed the RNN in nearly all configurations. For example, the best window size configuration (V₄ = 60) yielded an MSE of 2.18 for the LSTM compared to 6.81 for the RNN and 10.83 for the transformer. A similar pattern holds across hidden size, layers, and batch size. The transformer results were generally weaker than the other models, with all configurations producing MSE values above 7.0.

After identifying the local optimal hyperparameter value for each parameter, all models were trained using their respective optimal configurations from Table I. The LSTM achieved the best validation performance with an MAE of 1.20 and an MSE of 2.61, followed by the RNN with an MAE of 1.50 and an MSE of 3.75, and lastly the transformer with an MAE of 2.13 and MSE of 10.39. The baseline model achieved a validation MAE of 7.38 and MSE of 124.32. The ranking of the models is consistent with what was observed during the hyperparameter tuning process, where the LSTM and RNN both showed lower error regions while the transformer struggled to match their performance.

The best performing models were used to predict the next 200 data points. These were then compared to the test set. The results of which can be found in Table III. For the normal variant of the dataset we observe much larger errors than on

the validation set. Notably, we see an infinite error for the baseline model. This is due to the model recursively predicting larger and larger values, eventually predicting the integer limit, resulting in infinite differences between the prediction and the ground truth value. This makes it difficult to compare the other models to the baseline model.

The RNN outperforms the other models and its predictions are visualized in Figure 1. Here, we see the test set has large fluctuations at first, and then rapidly shrinks, similarly to the training data round index 600. Meanwhile, the RNN predictions slowly decrease, leading to a large MAE and MSE.

A. Individual Signal Extraction

The results of the models with the best hyperparameter configurations trained on the signal extraction variant of the dataset can be found in Table III. We observe that the RNN model has decreased in MAE performance, while all other models improved. Most notably is the transformer, which outperforms the others in MAE and is the only model that improved with the signal extraction method. However, the MSE shows that the RNN is still outperforming the other models, though it does not meaningfully improve compared to the baseline.

Figure 2 shows the transformer model’s future predictions. While it appears to closely match the signal in the test data, it is incorrectly offset such that it still accumulates a high error. However, this transformer seems to still better capture the global pattern of the train and test data better than the normal RNN based on visual inspection.

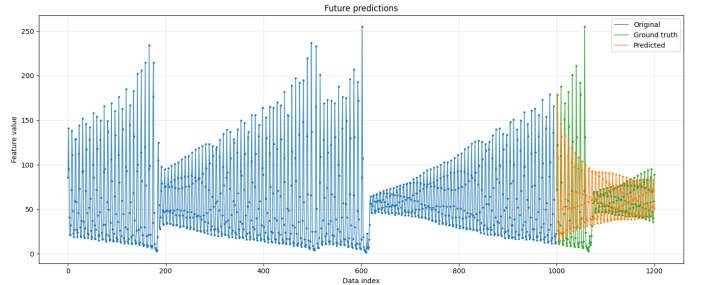


Fig. 1: Test results of the best performing **RNN**. Train dataset in Blue, with the future predictions in orange and the test dataset in green.

V. CONCLUSION & DISCUSSION

We aimed to predict future time-series values of a laser measurement signal dataset using tuned RNN, LSTM and transformer models. While our experimental results indicate high performance on the validation set, the test results remain poor. On the normal dataset, the RNN model performed best, with an MAE of 35.08 and MSE of 2128.7. This significantly outperformed the very poorly performing baseline model, but as indicated by Figure 1, the predicted signal does not seem to match the test data.

On the signal interpolated dataset, all trained models performed worse, except for the transformer, which improved in

TABLE II: MAE and MSE metrics for the **RNN**, **LSTM**, and **Transformer** models with different hyperparameter values. Bold text denotes the best value.

(a) RNN

	Window size		Hidden size		Layers		Learning rate		Batch size		Stride		Weight decay	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
V ₁	2.2600	18.4865	1.7011	4.5566	1.8833	5.3568	1.3630	3.1464	4.1236	27.9817	1.8195	5.2468	2.2765	8.3716
V ₂	2.3826	19.1097	1.6271	5.2815	1.7373	5.0059	1.5843	4.7146	2.3043	9.1650	2.2541	11.6503	1.5358	4.4744
V ₃	2.5051	14.2224	1.9041	6.8655	1.6722	4.8832	1.5429	4.4444	2.2955	10.0102	2.7276	18.8675	1.9884	7.1329
V ₄	1.9391	6.8112	1.6708	4.8354	1.6819	5.0234	1.8078	6.6782	2.0715	6.3679	6.7393	340.7627	1.4689	4.1732
V ₅	2.4196	25.3283	2.0485	6.8199	2.5673	11.0375	2.0070	8.3487	2.0350	7.5455	6.1302	102.3272	1.7973	5.7788

(b) LSTM

	Window size		Hidden size		Layers		Learning rate		Batch size		Stride		Weight decay	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
V ₁	1.5515	10.8673	1.1475	2.6117	1.1799	2.4536	2.4155	12.3822	1.3546	3.1725	1.1387	2.1672	1.1700	2.3171
V ₂	1.3408	3.9844	1.1228	2.4162	1.2620	2.5202	1.3200	3.4436	1.2863	2.9086	7.8392	516.0886	1.1739	2.4284
V ₃	1.6084	6.1976	1.1846	2.8053	1.2358	2.9068	1.1845	2.3464	1.4741	4.1224	6.7233	104.8139	1.1229	2.3484
V ₄	1.1254	2.1818	1.0514	2.1913	1.2544	3.0114	1.1774	2.6095	1.1634	2.3181	7.5093	143.7400	1.3308	3.1112
V ₅	1.5558	7.4027	1.1724	2.4877	1.5608	4.0880	1.2789	2.9794	1.2591	2.8820	4.0843	56.5745	1.6951	6.1859

(c) Transformer

	Window size		Hidden size		Layers		Learning rate		Batch size		Stride		Weight decay	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
V ₁	2.5844	14.1215	3.1003	17.6448	2.4822	11.6552	3.1622	25.7233	2.7642	15.2229	3.5690	25.4119	2.8152	16.3061
V ₂	3.0057	19.9563	2.6769	19.2807	2.5190	10.5478	1.9643	7.0207	3.0426	14.6751	11.0956	479.1272	2.2141	9.7710
V ₃	3.1670	21.0144	2.0873	7.2154	3.1167	16.4562	2.3565	10.6161	2.3428	10.2812	13.1188	423.5854	2.5990	11.9971
V ₄	2.5095	10.8349	2.4210	10.5163	2.6319	13.6655	3.0333	18.2659	3.1451	16.4883	7.7767	102.2892	2.7478	13.3808
V ₅	3.1347	28.5424	2.8332	17.5198	2.8943	16.0477	3.0132	16.8206	2.7733	16.5090	6.0039	56.4990	2.5595	11.6304

TABLE III: Test data results of best model configurations. The best results are highlighted in bold.

Variant		Baseline	RNN	LSTM	Transformer
Normal	MAE	∞	35.0784	54.1660	43.1495
	MSE	∞	2128.7	5150.6	3248.8
Signal Extraction	MAE	53.9994	51.7099	53.1575	42.4037
	MSE	4260.3	4258.3	4737.7	4332.6

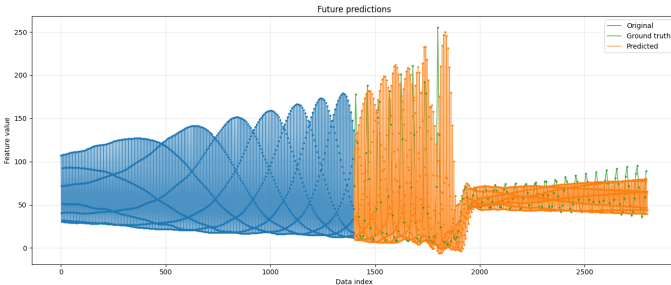


Fig. 2: Zoomed in results of the best performing **transformer** using the signal extraction variant of the dataset. Only the last 20% of training data is shown. Train dataset in Blue, with the future predictions in orange and the test dataset in green.

MAE. As indicated by Figure 2, this model seems to capture the test pattern. Yet, its predictions remain offset, resulting in a high error. Regardless, this was the only model we tested that was able to meaningfully capture both the wide and narrow signal patterns present in the dataset. Therefore, we believe this is still the best performing model, despite not having the lowest error.

We find even our best models to not be accurate enough for finely predicting future values of laser measurement signals, as the error is likely too large for any meaningful applications. We would advise future research to try expanding upon our signal interpolation approach, as this resulted in the most subjectively meaningful model (despite still having a large error). We believe our transformer model would be best suited to learn and capture the true data signal, if provided with enough training samples.

Lastly, we recognise that our results could be heavily influenced by our hyperparameter tuning approach. We acknowledge that different strategies, such as existing algorithms or a grid search would improve our ability to find an optimal combination of hyperparameters. We aim to experiment with, and apply, these algorithms in future iterations of this research. This would also allow us to include the transformer model hyperparameters.

REFERENCES

- [1] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in neural information processing systems* 30, 2017.

APPENDIX A

DETERMINING HYPERPARAMETER RANGES

For each of the seven hyperparameters, we determined a range of five values where the LSTM and RNN models might optimally perform. To determine these, we

started from hyperparameter values that indicated reasonable performance during initial model implementation. These initial values were: *window_size*=20, *hidden_size*=32, *number_of_layers*=2, *learning_rate*=0.001, *batch_size*=16, *stride*=1, and *weight_decay*=0.0001. We then iteratively tuned each one in turn, in order of estimated performance impact to the model: window size, hidden size, number of layers, learning rate, batch size, stride, and weight decay. For each hyperparameter, we visualised the MAE and MSE on both the training and validation set across a range of values. If the visualisation suggested that the error was still decreasing at the edges of our search window, we extended the range. If the range was too wide to clearly identify a low-error region, we reduced it. This was repeated until a range was found that minimized the errors for both models. Where MAE and MSE disagreed on the optimal range, we used MSE as the deciding factor, as it penalises larger errors more aggressively, making it a criterion that shows poor performance on predictions more clearly compared to MAE. The hyperparameters were tuned sequentially in the order of importance we defined before. Once a good range was found for a hyperparameter, its local optimum in that range was used as the value when tuning the next hyperparameter. e

It should be noted that this approach has limitations when the two models disagree on the optimal range, as a range that is suitable for one model may not be optimal for the other. In such cases, we selected the range that minimized the errors for both models, which could result in a suboptimal range for one or both models.

To illustrate our procedure, consider Figure 6. We initially evaluated layer counts between 1 and 10 with a step size of 1. Since the error clearly worsened before reaching the upper boundary of 10, there was no need to further extend the range. The best performance we can observe in the figure is found between 2 and 4 layers, while after 6 layers the error of the LSTM rapidly increases. Based on this, we identified that 1 and 5 should be the lower and upper bounds of our value range with a step size of 1 between them for the rest of our values.

The same method was applied to the other six hyperparameters. The resulting plots can be seen in Figures 3-9. The final ranges selected for the hyperparameter values can be found in the main report in Table I.

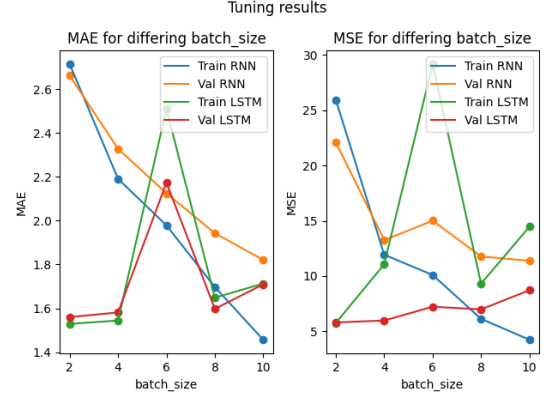


Fig. 3: Example of a tuning result graph for the number of batches.

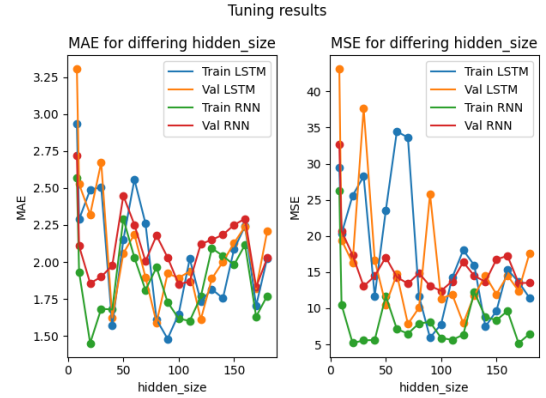


Fig. 4: Example of a tuning result graph for the hidden size.

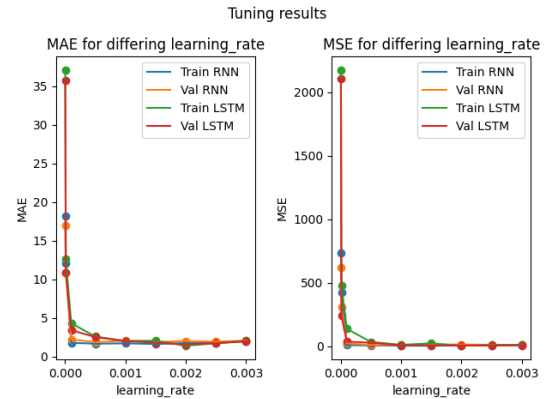


Fig. 5: Example of a tuning result graph for the learning rate.

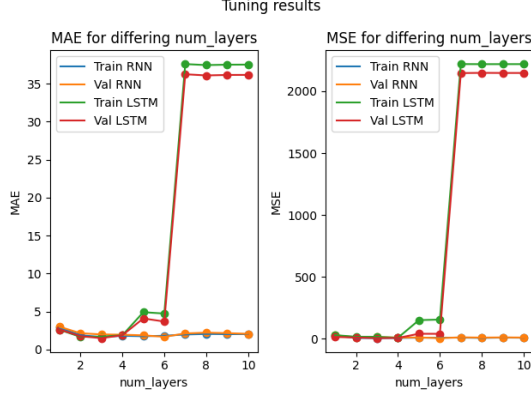


Fig. 6: Example of a tuning result graph for the number of layers.

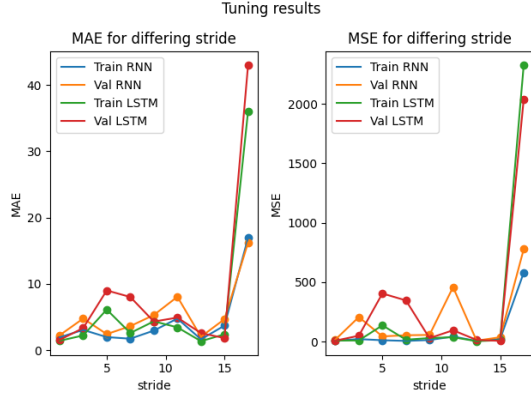


Fig. 7: Example of a tuning result graph for the stride.

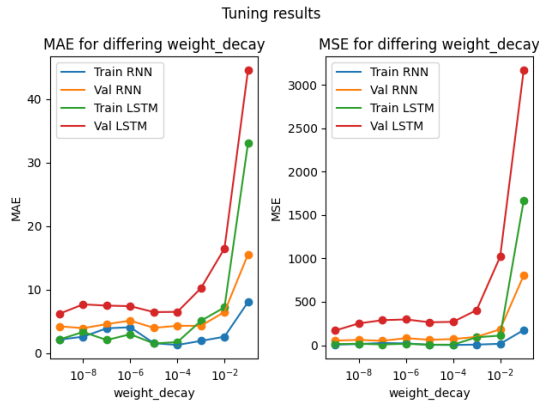


Fig. 8: Example of a tuning result graph for the weight decay.

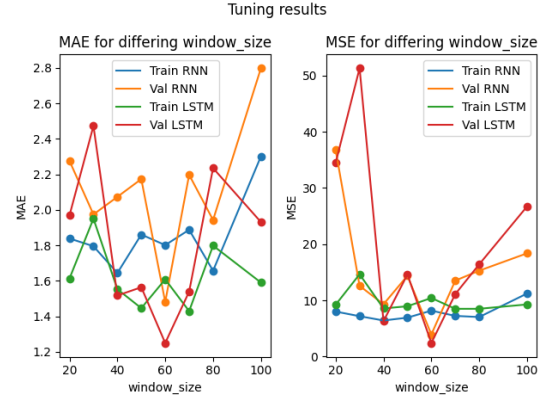


Fig. 9: Example of a tuning result graph for the window size.

APPENDIX B EVALUATION METRICS

To evaluate model performance, we used two metrics: Mean Absolute Error (MAE) and Mean Squared Error (MSE). Both metrics measure the average deviation of the model's prediction from the ground truth values.

The MAE is defined as

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

where n is the number of data points, y_i is the i^{th} ground truth value, and $\hat{y}_i$ is the i^{th} predicted value. The MAE computes the average of the absolute differences between prediction and ground truth values. This means a deviation of 2 is penalised exactly twice as much as a deviation of 1.

The MSE is defined as

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

where n is the number of data points, y_i is the i^{th} ground truth value, and $\hat{y}_i$ is the i^{th} predicted value. The MSE computes the average of the squared differences between prediction and ground truth values. Squaring the errors means that larger deviations contribute more to the final error, for example a deviation of 2 is penalised four times as much as a deviation of 1, making the MSE more sensitive to outliers when comparing it to MAE.

APPENDIX C INDIVIDUAL SIGNAL WAVES

During our data exploration phase, we visualised the data points in a scatterplot, as seen in Figure 10. Visual inspection of the original signal reveals what appears to be multiple distinct wave-like patterns overlaid on top of one another (e.g., around point 200-350), suggesting the data may not originate from a single continuous source but rather from several alternating signals recorded in a round-robin fashion. To investigate this, a de-interleaving approach was applied. Our thought was that, if n separate signals were recorded by

cycling through each source in order, then the original signal waves could be decomposed by extracting every n^{th} data point. So, signal X is reconstructed using indexes $[i, i + 1n, i + 2n, \dots]$, yielding n sub-signals each of approximately $\frac{1000}{n}$ data points.

An important parameter to set for this method to work is how many waves there are, denoted by n . We set this parameter by counting how many waves there are in Figure 10. We then also tried similar values around this n , concluding the best separation was $n = 7$ waves, because the different signals that could be manually identified are perfectly separated. An $n = 7$ leaves each wave with approximately 143 data points. A visualisation of the individual signals can be found in Figure 11.

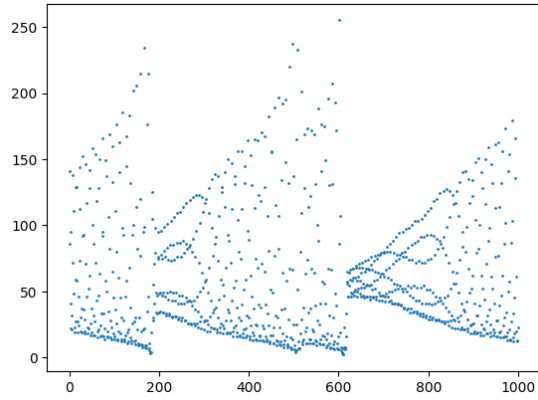


Fig. 10: All training data points visualised in a scatter plot.

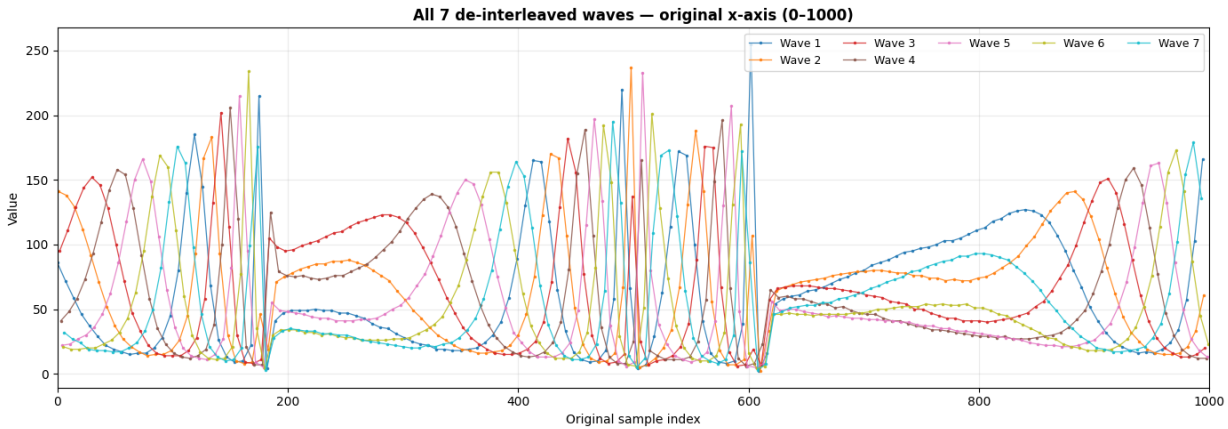


Fig. 11: Visualisation showing the 7 different signals on the training data.